

PHASE 4

CACHING

High-End System Design Interview Notes

THE BIG IDEA

Caching keeps frequently needed data closer to the application so repeated requests can be served with lower latency and less database work.

Application → Cache → (miss) → Database → Cache → Application

Phase 4 Learning Map

1. Why caching exists
2. Cache hit vs cache miss
3. Redis basics
4. Cache-aside
5. Write-through
6. Write-back
7. TTL
8. LRU and eviction
9. Cache invalidation
10. Hot keys
11. Cache stampede
12. Complete request pattern + interview trade-offs

INTERVIEW MINDSET

Caching is not simply “put Redis in front of the database.” You must explain what is cached, why it is safe to cache, how it expires, how it is invalidated, and what happens during failure or heavy traffic.

1. Why Caching Exists

A cache is a fast storage layer that keeps data likely to be requested again. The goal is usually lower latency and reduced load on slower or more expensive systems.

Why add a cache?

- ⚡ Lower latency — memory access can be much faster than repeatedly querying a database.
- 📉 Reduce database load — repeated reads can be served without hitting the database every time.
- 📈 Improve throughput — the system can handle more read requests with the same database capacity.
- 💰 Reduce cost — fewer expensive database operations can reduce infrastructure pressure.
- 🌐 Improve scalability — a distributed cache can serve high-volume read traffic across application instances.

IMPORTANT

Caching adds complexity. Cached data can become stale, disappear, or become unavailable. A cache should normally be treated as an optimization layer, not blindly as the only source of truth.

2. Cache Hit vs Cache Miss

Every cache lookup has a basic decision: is the requested key already present?

Cache Hit

- Requested key exists in the cache.
- Application returns the cached value.
- Database is avoided for that request.

Cache Miss

- Requested key is not in the cache, or the entry has expired/been evicted.
- Application fetches the data from the database or another source.
- Application stores the result in the cache for future requests.

CACHE HIT RATIO

Hit ratio = $\text{cache hits} \div \text{total cache requests}$. A higher hit ratio often means the cache is serving more requests, but a high hit ratio alone does not prove the system is well designed.

3. Redis Basics

Redis is an in-memory data store commonly used for caching, sessions, counters, rate limiting, queues, and other fast-access workloads.

- Data is primarily kept in memory for fast access.
- Supports structures such as strings, hashes, lists, sets, sorted sets, and more.
- Keys identify values, making direct lookups natural.
- Supports expiration/TTL for keys.
- Can be deployed as a shared cache accessible by multiple application instances.

```

SET user:101 "Harshith"
GET user:101

SET session:abc "user-101" EX 3600
TTL session:abc

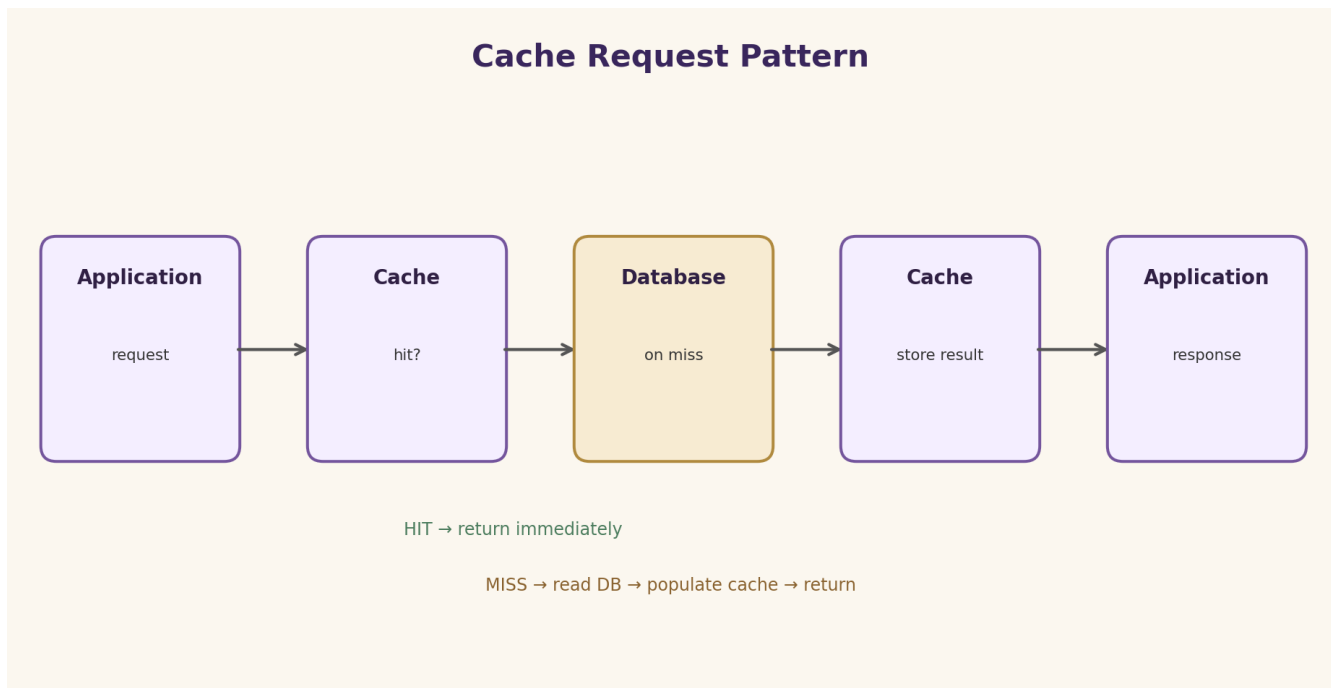
```

SYSTEM DESIGN NOTE

When several application servers need to share the same cache, a centralized/distributed Redis deployment can prevent each server from having an isolated local cache.

4. Cache-Aside Pattern

Cache-aside is one of the most common caching patterns. The application controls cache reads and writes.



13. Application receives a request.
14. Application checks the cache.
15. If HIT → return the cached value.
16. If MISS → query the database.
17. Store the database result in the cache.

18. Return the result to the application/client.

```
value = cache.get(key)

if value is None:
    value = database.get(key)
    cache.set(key, value, ttl=300)

return value
```

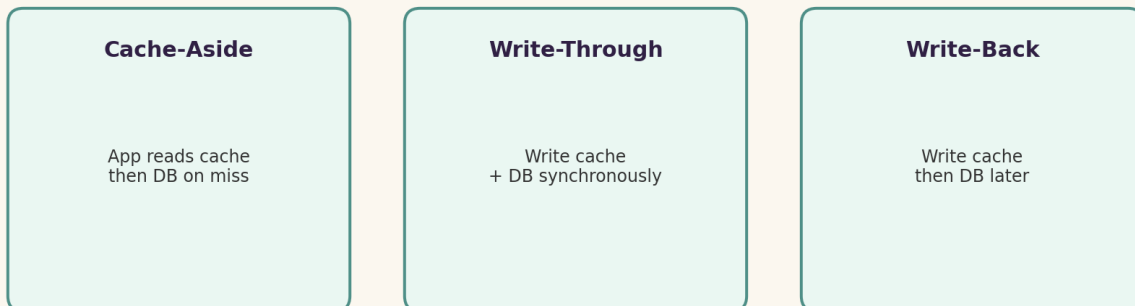
WHY IT IS POPULAR

It is simple, flexible, and lets the application decide exactly what should be cached. The main challenge is handling stale data and cache misses correctly.

5. Write-Through Caching

In write-through caching, a write goes through the cache and the cache synchronously updates the underlying database as part of the write flow.

Common Cache Write Strategies



- Application writes data.
- Cache receives the write.
- Cache updates the database synchronously.
- After the write completes, cache and database are intended to be aligned according to the system's guarantees.

TRADE-OFF

Reads can benefit from warm cache data, but writes may have higher latency because the database update is part of the write path.

6. ⚡ Write-Back Caching

In write-back caching, the application writes to the cache first, and the database update happens later/asynchronously.

- Fast write path because the initial write can finish without waiting for the database.
- Useful for workloads where high write throughput is important.
- More failure complexity: if the cache fails before data is persisted, data can be lost unless the architecture provides durable recovery mechanisms.
- Requires careful ordering, retries, persistence, and monitoring.

INTERVIEW WARNING

Do not describe write-back as “always better.” It trades some durability/simplicity for faster writes and higher asynchronous complexity.

7. ⌚ TTL — Time To Live

TTL is the amount of time a cache entry is allowed to remain valid before it expires.

```

SET product:42 '{"...}" EX 300
# Entry expires after 300 seconds

```

Why TTL is useful

- Prevents stale entries from living forever.
- Automatically cleans up old cache data.
- Provides a simple freshness boundary.
- Helps protect against unbounded cache growth.

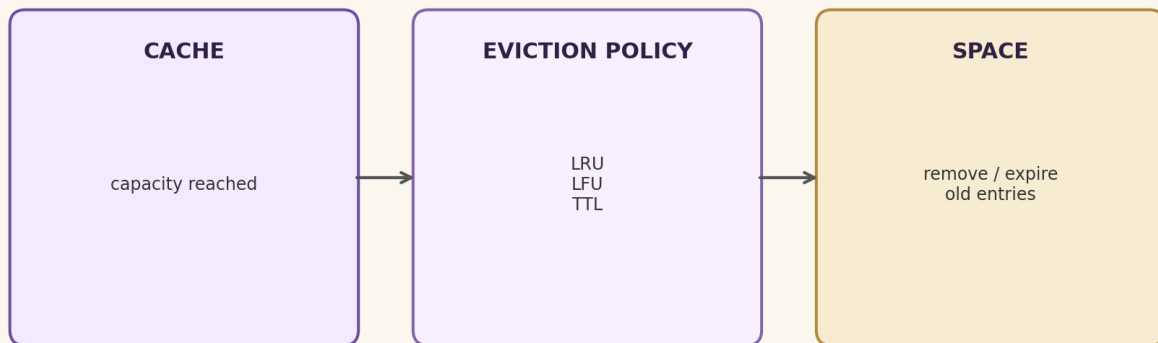
TTL IS NOT A MAGIC STALE-DATA SOLUTION

A TTL is only one freshness mechanism. If important data changes immediately, waiting for TTL expiration may be too slow. Explicit invalidation may be required.

8. 🗑️ LRU and Cache Eviction

A cache has limited memory. When it becomes full, an eviction policy determines which entries should be removed.

Eviction: What Happens When Cache Fills?



LRU — Least Recently Used

- Evicts entries that have not been used recently.
- Works well when recently accessed data is more likely to be requested again.
- Common mental model: keep the items people are actively using; remove older unused entries when space is needed.

Other eviction ideas

- LFU — Least Frequently Used: consider how often an entry is accessed.
- TTL-based expiration: remove entries after their lifetime.
- Database/cache products may provide several configurable policies.

INTERVIEW POINT

Eviction is different from invalidation: eviction removes data because the cache needs space or a policy says it should disappear; invalidation removes/marks data because the cached value is no longer valid.

9. 🔑 Cache Invalidation

Cache invalidation means ensuring a cached value is removed or updated when the underlying data changes.

Common approaches

Approach	How it works	Trade-off
TTL	Let the entry expire automatically	Simple, but stale data may remain until expiry
Delete on write	Remove affected cache key when source data changes	Simple but requires correct key tracking
Update on write	Update cached value when source changes	Can keep cache fresh but adds write complexity
Versioned keys	Use a version in the key so old data becomes unreachable	Useful in some architectures, but requires key management

THE HARD PART

Cache invalidation is difficult because every code path that changes the source data must consider what cached representations may now be stale.

10. Hot Keys

A hot key is a cache key that receives a disproportionately large amount of traffic.

- Example: a celebrity's profile, a viral post, or a highly viewed product.
- A single key can become a bottleneck even when the overall cache has plenty of capacity.
- Hot keys can overload one cache node or create concentrated network/CPU pressure.

Possible strategies

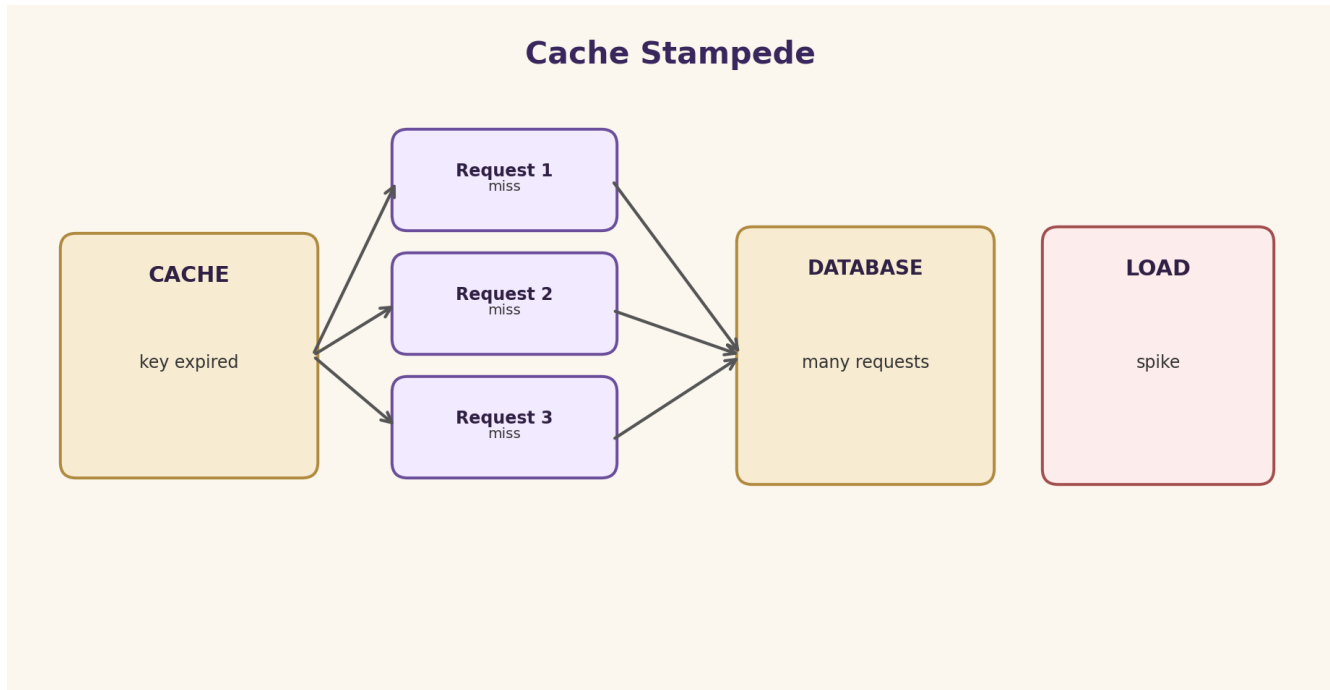
- Replicate/cache the hot value across multiple cache nodes where appropriate.
- Use local in-process caching for extremely hot, safe-to-cache data.
- Add request coalescing/single-flight so many concurrent misses do not all perform the same expensive work.
- Review whether the key can be partitioned safely.

DESIGN QUESTION

Ask: “What happens if 100,000 users request the exact same key at the same time?” That question often exposes hot-key and stampede problems.

11. Cache Stampede

A cache stampede occurs when many requests miss the same cache entry around the same time and all hit the underlying database or service.



- Popular key expires.
- Thousands of requests arrive.
- All see a miss.
- All query the database.
- Database suddenly receives a traffic spike.

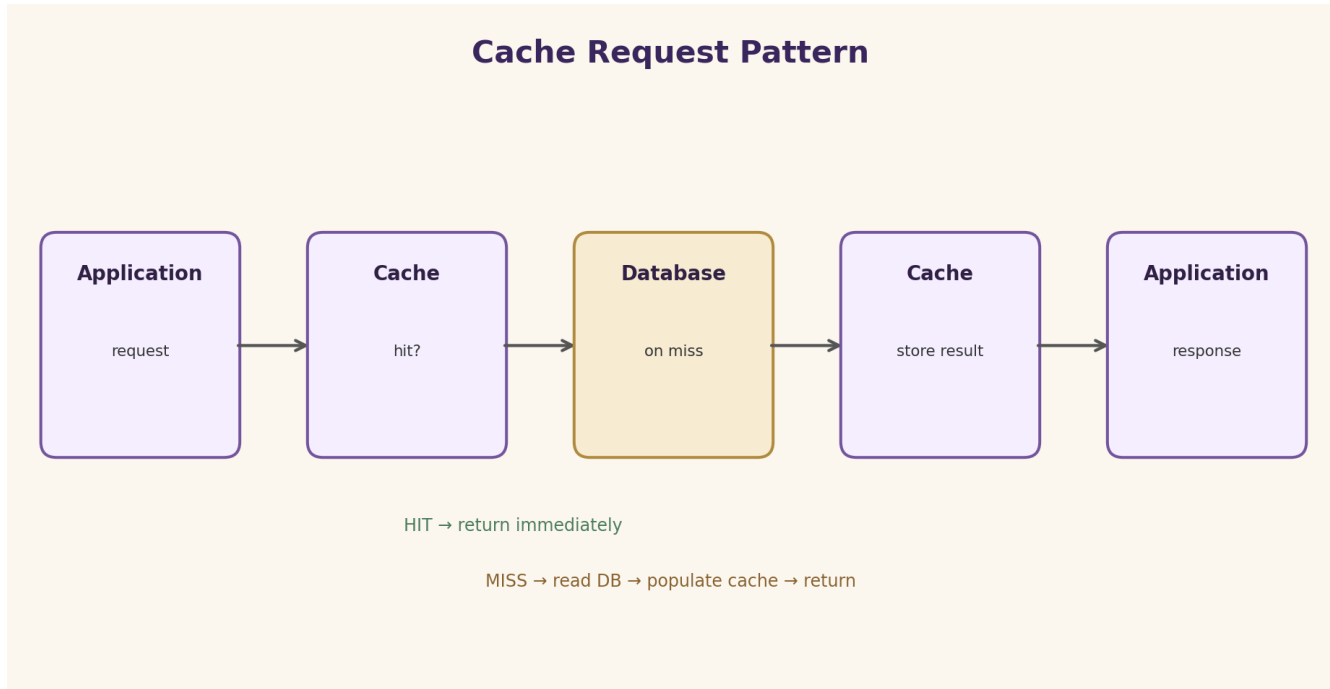
Ways to reduce the risk

- Request coalescing / single-flight — allow one request to rebuild the value while others wait or reuse the result.
- Jittered TTLs — add small random variation to expiration times to avoid synchronized expiry.
- Early refresh — refresh popular entries before they expire.
- Stale-while-revalidate — serve an acceptable stale value while one process refreshes it, when the business requirement permits.
- Rate limiting and load protection — prevent a sudden cache miss from overwhelming the origin.

KEY DISTINCTION

Hot key = too many requests concentrate on one key. Cache stampede = many requests simultaneously miss and overwhelm the origin. They can happen together, but they are not identical.

12. Complete Caching Pattern



THE PATTERN TO REMEMBER

Application → Cache → HIT → Application

Application → Cache → MISS → Database → Cache → Application

Step-by-step

19. Client sends request to application.
20. Application checks cache using a key.
21. Cache HIT → cached response/data is returned.
22. Cache MISS → application queries database.
23. Database returns the data.
24. Application stores the result in cache with an appropriate TTL.
25. Application returns the result to the client.

13. Caching Trade-offs

Benefit	Cost / Risk	Design Question
Lower latency	Stale data	How fresh must the data be?
Less DB load	Cache misses	What happens on a miss?
Higher throughput	Memory cost	Which data deserves cache

Better scalability	Invalidation complexity	How will writes invalidate/update keys?
Fast reads	Hot keys	Can one key overload the cache?
Reduced origin traffic	Stampede	What happens when popular keys expire?

14. Interview Questions & Answers

Q: Why do we need caching?

A: To reduce latency and repeated work on the database/origin. It can improve throughput and scalability, but introduces freshness and invalidation complexity.

Q: What is a cache hit?

A: The requested key exists and can be served from the cache.

Q: What is a cache miss?

A: The requested key is unavailable in the cache, so the application retrieves it from the source and may populate the cache.

Q: Explain cache-aside.

A: Application checks cache first; on miss it reads the source, stores the result in cache, then returns it.

Q: Write-through vs write-back?

A: Write-through updates the underlying store synchronously through the cache path; write-back acknowledges the cache write and persists to the underlying store later.

Q: What is TTL?

A: Time To Live: how long a cache entry is allowed to remain before expiration.

Q: What is LRU?

A: Least Recently Used: evict entries that have not been accessed recently.

Q: What is cache invalidation?

A: Removing or updating cached data when the source value changes.

Q: What is a hot key?

A: A cache key receiving unusually concentrated traffic.

Q: What is a cache stampede?

A: Many requests miss an important key together and overwhelm the origin with duplicate work.

15. One-Page Caching Cheat Sheet

Concept

Remember

Cache	Fast layer holding frequently requested data.
Hit	Data found in cache.
Miss	Data not found; read source and optionally populate cache.
Redis	Popular in-memory data store used for caching and more.
Cache-Aside	App reads cache; on miss reads DB and fills cache.
Write-Through	Write path updates cache and DB synchronously.
Write-Back	Write cache first; persist to DB later.
TTL	Automatic expiration lifetime.
LRU	Evict least recently used entries.
Invalidation	Remove/update cached data because source changed.
Hot Key	One key receives concentrated traffic.
Stampede	Many simultaneous misses overload the origin.
Core flow	App → Cache → miss → DB → Cache → App

16. Practice Before Moving to Phase 5

26. Why is a cache usually faster than a database?
27. Explain cache hit and cache miss with an example.
28. Explain Redis in simple terms.
29. Draw the cache-aside flow from memory.
30. When would write-through be useful?
31. When would write-back be risky?
32. Why do we use TTL?
33. LRU vs TTL — what is the difference?
34. What is cache invalidation and why is it difficult?
35. What is a hot key?
36. What is a cache stampede?
37. How would you protect a database if a popular cached key expires?
38. How would you decide what data should be cached?
39. What happens if Redis goes down?

MASTER QUESTION

If the interviewer says: “Our database is overloaded because millions of users repeatedly request the same product,” explain your cache design from key choice → hit/miss → TTL → invalidation → hot-key protection → stampede protection → failure handling.

PHASE 4 COMPLETE ✓

Cache the right data • Keep it fresh enough • Protect the origin • Design for failure